

8

Laboratory 8

8	Object tracking	75
8.1	Correlation Filter	
8.2	Correlation Filter – exercise	
8.3	Tracking with Siamese neural network	
8.4	Siamese network – exercise	

8. Object tracking

8.1 Correlation Filter

One approach to the tracking problem is to use methods from the correlation filter family. In these algorithms a certain filter is used with which the object to be tracked is represented. In order to predict the new position of the object, a correlation is performed between the tracking area and the filter. The calculation is performed in the frequency domain, as it is faster to calculate the Fourier transform with the *Fast Fourier Transform* algorithm and use the convolution theorem than to calculate the correlation directly. The tracking algorithms of the correlation filter family have low computational complexity (i.e. real-time tracking is possible) while maintaining competitive tracking quality.

The simplest version of the algorithm described in the paper will be presented: [Visual Object Tracking using Adaptive Correlation Filters](#).

In the first frame of the image, the filter is initialised based on the greyscale image patch f that contains the object to be tracked.



Figure 8.1: Selected object to track.

$$H_t^* = \frac{A}{B} \quad (8.1)$$

$$A = \sum_{i=1}^N G_i \odot F_i^* \quad (8.2)$$

$$B = \sum_{i=1}^N F_i \odot F_i^* \quad (8.3)$$

where F_i is the discrete Fourier transform of the image patch f_i , while G is the Fourier transform of the two-dimensional Gaussian distribution g of the size of the tracked area.

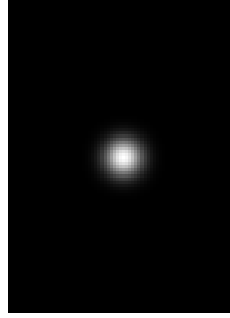


Figure 8.2: Example of a discrete 2D Gaussian distribution g with the size of the tracked region.

The operation $\odot$ denotes element-by-element multiplication, and X^* denotes complex conjugate (the Fourier transform results are complex-valued).

The sum over i means that from one object sample in the first tracking frame, a set of N training samples consisting of random rotations of the base sample should be created.

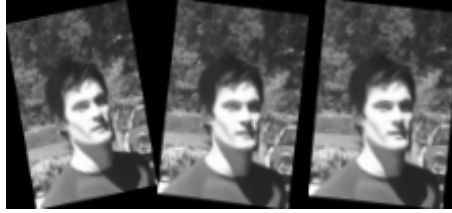


Figure 8.3: Training samples.

Then, a position prediction is made in each image frame:

$$\mathcal{F}^{-1}\{F \odot H^*\} \quad (8.4)$$

where $\mathcal{F}^{-1}$ denotes the inverse discrete Fourier transform, while F here is the transform of the image patch centred on the last known position of the object. The correlation map obtained in this way represents the displacement of the object with respect to the previous image frame, as predicted by the filter.

After each prediction, the filter is updated with a portion of the current image frame cropped at the new object position:

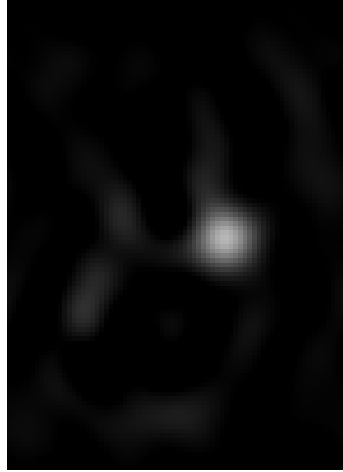


Figure 8.4: Correlation map.

$$A_t = \eta G \odot F_t^* + (1 - \eta) A_{t-1} \quad (8.5)$$

$$B_t = \eta F_t \odot F_t^* + (1 - \eta) B_{t-1} \quad (8.6)$$

where $\eta \in [0, 1]$ is the assumed learning rate.

8.2 Correlation Filter – exercise

Exercise 8.1 The purpose of the exercise is to learn a method that uses correlation filters to track objects.

1. Download the materials and the exercise template from the UPeL platform.
2. The exercise has been prepared in a Jupyter notebook. The file `.ipynb` can be opened and run using Google Colab (colab.research.google.com). It is also possible to run the file using the VS Code editor. You will probably be required to install the appropriate libraries. If we choose VS Code, we skip loading the library:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. Follow the instructions in the file.
4. The solution to the task is the completed and working `.ipynb` file, which should be sent to the UPeL platform.

8.3 Tracking with Siamese neural network

A Siamese network is a Y-shaped network with two branches joined to produce a single output. Irrespective of the branches' structure, a Siamese network can be considered as a similarity function that measures the resemblance between two inputs. We can define the basic Siamese architecture using Equation (8.7), where ϕ stands for extracting deep features (e.g. via network's branches), z represents the template (e.g. exemplar, object to track), x represents the patch that is compared to the exemplar (e.g. search region, ROI) and γ is a cross-correlation.

$$y = \gamma(\phi(z), \phi(x)) \quad (8.7)$$

Its scheme is presented in Fig. 8.5. The input z represents the tracked object selected in the first frame and scaled to 127×127 pixels. In the next frames, the ROI around the previous object's location is selected and presented to the network as the input x , scaled to 255×255 pixels. Both patches (exemplar and ROI) result in two blocks of feature maps, of sizes $17 \times 17 \times 32$ (for z) and $49 \times 49 \times 32$ (for x) respectively. Finally they are cross-correlated and a heat map of size 33×33 is obtained. Its highest peak indicates the object's location. The ROI is selected approximately 4 times greater than object's size, over five scales to handle object's size variations.

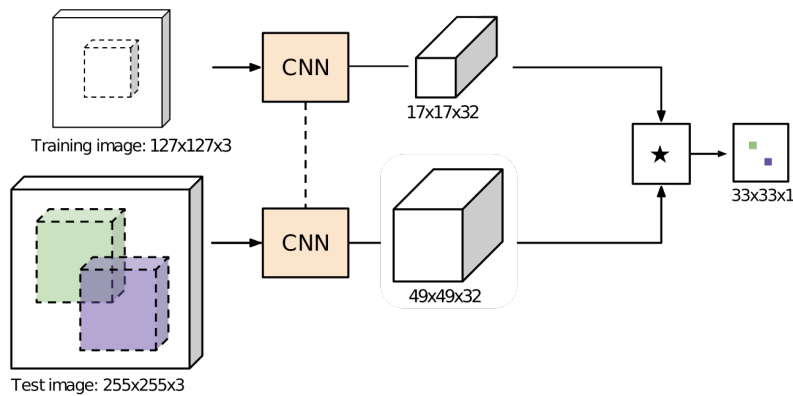


Figure 8.5: A fully-convolutional Siamese Network for tracking.

The original description of the algorithm can be found at: [Fully-Convolutional Siamese Networks for Object Tracking](#).

8.4 Siamese network – exercise

Exercise 8.2 The purpose of the exercise is to learn about the architecture of a Siamese neural network for object tracking. This is an example in which a neural network will be used.

1. Download the materials and the exercise template from the UPeL platform.
2. The exercise has been prepared in Jupyter notebook. The file `.ipynb` can be opened and run using Google Colab (colab.research.google.com). It is also possible to run the file using the VS Code editor. You will probably be required to install the appropriate libraries. If we choose VS Code we skip loading the library:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. Follow the instructions in the file.
4. The solution to the task is the completed and working `.ipynb` file, which should be sent to the UPeL platform.